

PORTFOLIO PROJECT BUILD MANUAL

Data Engineering Concepts Handbook

[Databricks](#) · [Apache Spark](#) · [Delta Lake](#) · [Unity Catalog](#)

A topic-by-topic reference explaining what each component is, why it exists, how it works, and the technical points that matter in practice — written to be read for understanding rather than memorised.

This document contains

- Topic-by-topic explanations across the Databricks stack
- What each thing is, why it exists, and how it works
- The technical details that matter in production

Vivek Pandey — Data Engineer

[Azure](#) · [Databricks](#) · [PySpark](#) · [Delta Lake](#)

vp171097.github.io/DevWebsite

A topic-by-topic explanation of what each thing is, why it exists, how it works, and the technical points that matter in practice.

1. Apache Spark

What it is

A distributed compute engine. You write code as though you're operating on one table; Spark splits the data into partitions, runs the same operation on each partition in parallel across many machines, and combines the results.

Why it exists

A single machine runs out of memory and CPU long before modern data volumes are exhausted. Spark's predecessor, MapReduce, wrote intermediate results to disk between every step. Spark keeps intermediate data in memory where possible and optimises the whole pipeline before running it, which makes it an order of magnitude faster for most workloads.

The architecture

Driver — the coordinator. Holds the SparkSession, builds the execution plan, splits it into tasks, schedules them, and collects results. It also runs any non-Spark code you write (plain Python, pandas, API calls). If the driver dies, the whole application dies.

Cluster manager — allocates machines. On Databricks this is the control plane; elsewhere it might be YARN or Kubernetes.

Executors — JVM processes on worker nodes that actually do the work. Each executor has a number of cores, and each core runs one task at a time. Executors also hold cached data. If an executor dies, Spark re-runs its tasks elsewhere.

```
Cluster
├── Driver node (1) → SparkContext, scheduler, your Python process
└── Worker nodes (N) → each runs 1 Executor JVM
                        ├── Executor has C cores (task slots)
                        │   ├── each core runs 1 Task
                        │   └── 1 Task = 1 Partition of data
```

On Databricks, one worker node runs exactly one executor, and executor cores equal the VM's vCPUs. Total parallelism is workers × cores per worker.

The execution hierarchy

Term	Meaning
Job	Everything triggered by one action (<code>write</code> , <code>count</code> , <code>display</code>)
Stage	A group of tasks that run without a shuffle. A shuffle boundary creates a new stage
Task	The smallest unit of work — one task processes one partition on one core
Partition	A chunk of the data. The unit of parallelism

Lazy evaluation

Transformations (`filter` , `select` , `join`) don't execute — they build a plan. Only an action triggers execution. This is deliberate: by deferring, Spark sees the whole pipeline at once and can optimise globally — pushing filters down into the file scan, eliminating columns you never reference, choosing join strategies.

Narrow vs wide transformations

Narrow — each output partition depends on one input partition. No data movement. `filter` , `select` , `withColumn` , `union` .

Wide — an output partition needs data from many input partitions, so data must move across the network. `groupBy` , `join` , `distinct` , `orderBy` , window functions.

Wide transformations cause **shuffles**, which are the dominant cost in most Spark jobs.

The shuffle

Redistributing data so rows with the same key land in the same partition. Mechanically: each map task writes its output to local disk, split by target reducer; each reduce task then fetches its blocks over the network.

The cost is serialisation, disk I/O, network transfer, potential spill to disk if it doesn't fit in memory, and a hard barrier — the next stage cannot start until the shuffle completes.

Most Spark optimisation work is really about avoiding or shrinking shuffles.

Catalyst optimizer

Spark's query optimiser. Four phases:

1. **Analysis** — resolve column and table names against the catalog
2. **Logical optimisation** — rule-based rewrites: predicate pushdown, column pruning, constant folding, projection collapsing
3. **Physical planning** — generate candidate physical plans, choose one (mainly join strategies)
4. **Code generation** — Tungsten compiles the plan into JVM bytecode

Tungsten and whole-stage codegen

Tungsten is the memory and CPU efficiency layer: off-heap binary storage (no JVM object overhead), cache-aware sorting and hashing. Whole-stage codegen fuses several operators into a single generated Java function, so rows stay in CPU registers rather than being pulled through an iterator per operator.

Adaptive Query Execution (AQE)

Re-optimises at runtime using actual shuffle statistics rather than pre-execution estimates. Three capabilities:

- **Coalescing shuffle partitions** — merges many tiny partitions into a sensible number
- **Skew join handling** — detects oversized partitions and splits them
- **Dynamic join strategy switching** — converts a sort-merge join to a broadcast join when the real size turns out to be small

Enabled by default on modern runtimes (`spark.sql.adaptive.enabled`).

Join strategies

Strategy	When chosen	Cost
Broadcast Hash Join	One side below <code>autoBroadcastJoinThreshold</code> (default 10 MB) or hinted	Cheapest — no shuffle
Shuffle Sort Merge Join	Two large sides, sortable keys. The default	Shuffles and sorts both sides
Shuffle Hash Join	One side much smaller but not broadcastable	Shuffles, builds hash table per partition
Broadcast Nested Loop	Non-equi join with a small side	Expensive
Cartesian	No join condition	Almost always a bug

Executor memory model

```
JVM Heap (spark.executor.memory)
├─ Reserved (~300 MB, fixed)
├─ Usable
│   ├── Unified Memory (spark.memory.fraction, default 0.6)
│   │   ├── Execution → shuffles, joins, sorts, aggregations
│   │   └── Storage → cached DataFrames, broadcast variables
│   └── User Memory (0.4) → your data structures, UDF objects

```

Outside the heap:

```
├─ spark.executor.memoryOverhead → Python worker processes,
   off-heap allocations, network buffers.
   Default = max(384 MB, 10% of executor memory)
```

The rule: execution memory can evict storage memory; storage can never evict execution. Spark sacrifices your cache to complete a shuffle, never the reverse.

PySpark implication: Python workers live outside the heap, funded by `memoryOverhead`. A "container killed for exceeding memory limits" error or exit code 137 means raise overhead, not `executor.memory`.

RDD vs DataFrame

RDDs are the original low-level API — typed JVM objects, no schema, no Catalyst, no Tungsten. DataFrames are schema-aware distributed tables that go through the full optimisation stack. In PySpark this distinction is critical. DataFrame operations become a plan executed entirely in the JVM — Python never touches individual rows. RDD operations and Python UDFs serialise every row across the JVM-Python boundary, which is dramatically slower.

2. Partitioning, Skew, and Spill

Partitions

The unit of parallelism. Too few and each task handles too much data (spill, OOM, underused cores). Too many and scheduling overhead dominates and you produce tiny output files.

Target roughly 128 MB per partition. For 10 GB that's about 80 partitions.

Key settings:

- `spark.sql.files.maxPartitionBytes` (default 128 MB) — read partition size
- `spark.sql.shuffle.partitions` (default 200) — post-shuffle partition count

The default of 200 is arbitrary and usually wrong. For 1 GB it means 5 MB partitions; for 5 TB it means 25 GB partitions.

Non-splittable files (gzip, single-object JSON) get exactly one partition regardless of size — a 5 GB gzip runs on one core.

repartition vs coalesce

	<code>repartition(n)</code>	<code>coalesce(n)</code>
Shuffle	Yes, full	No
Direction	Up or down	Down only
Result	Even partitions	Can be uneven
Upstream effect	None	Propagates reduced parallelism upstream

`coalesce(1)` before a heavy transformation makes that transformation single-threaded — a common trap.

`repartition("col")` redistributes in memory for the current job. `.write.partitionBy("col")` writes physical directories on disk. Different things, often used together.

Data skew

When one key holds far more data than others, one task does most of the work while the rest finish and idle. Adding workers does nothing, because the single oversized task still runs on one core.

Detection: Spark UI → Stages → task summary. Max duration or max shuffle read orders of magnitude above the median.

Fixes, in order:

1. AQE skew join (`spark.sql.adaptive.skewJoin.enabled`)
2. Broadcast the small side — no shuffle means skew stops mattering
3. Filter null keys — nulls all hash to one partition

4. Salting — add a random suffix to the hot key, explode the other side across the same range, join on key+salt, aggregate the salt away
5. Isolate hot keys and process them separately

Spill

When execution memory runs out during a shuffle, sort, or aggregation, Spark writes to local disk. Visible in the Stages tab as `Spill (Memory)` and `Spill (Disk)`.

Fixes: more partitions (smaller each), more memory per core, less shuffling, fix skew.

The small files problem

Every file is at least one task. Thousands of small files mean scheduling overhead, expensive directory listing, poor compression, and ineffective column pruning — the job spends more time on metadata than data.

Fixes: `optimizeWrite` and `autoCompact` on Delta tables, scheduled `OPTIMIZE`, avoid partitioning on high-cardinality columns, and use Auto Loader with file notifications rather than directory listing.

3. Delta Lake

What it is

An open table format: Parquet data files plus a transaction log. That log is what turns a directory of files into a real table with ACID guarantees.

Why it exists

Plain Parquet on object storage has no transactions. A failed write leaves partial files. Concurrent writers corrupt each other. There's no way to update or delete a row. There's no history. Delta fixes all of that without a database server.

The transaction log

```
my_table/
├─ part-00000-....snappy.parquet      ← plain Parquet data
├─ part-00001-....snappy.parquet
└─ _delta_log/
    ├─ 00000000000000000000000000000000.json      ← commit 0
    ├─ 00000000000000000000000000000001.json      ← commit 1
    ├─ 00000000000000000000000000000010.checkpoint.parquet  ← every 10 commits
    └─ _last_checkpoint
```

Each commit is an ordered JSON file containing **actions**:

Action	Contents
<code>metaData</code>	Schema, partitioning, table properties
<code>protocol</code>	Minimum reader and writer versions required
<code>add</code>	A file added, plus statistics: min, max, null count, row count
<code>remove</code>	A tombstone marking a file as no longer part of the table
<code>commitInfo</code>	Operation type, user, timestamp, metrics
<code>txn</code>	Idempotency marker for streaming writes

Atomicity comes from the commit being a single atomic file creation — it either exists or it doesn't. Readers replay the log to reconstruct table state, so they always see a consistent snapshot.

Statistics in the `add` actions power file skipping (only read files whose min/max range could match your filter) and make `COUNT(*)` effectively instant, since row counts are already in the log.

Checkpoints every 10 commits consolidate state into Parquet so readers don't replay from zero.

Optimistic concurrency control

A writer reads the current version, does its work, then attempts to commit as the next version. If someone committed first, it re-reads, checks whether that commit touched the same files or partitions, and either retries transparently or throws.

`ConcurrentAppendException` typically means a MERGE ran while something appended into the same partition. The fix is to make write scopes disjoint and to include the partitioning column in the condition, so Delta can see that the write is scoped.

Time travel

Because every version is recorded, you can query the past:

```
SELECT * FROM t VERSION AS OF 42;
SELECT * FROM t TIMESTAMP AS OF '2026-08-15';
RESTORE TABLE t TO VERSION AS OF 42;
DESCRIBE HISTORY t;
```

RESTORE creates a **new commit** pointing at the old file set, so the restore itself is auditable and reversible.

How far back you can go is bounded by two properties:

- `delta.logRetentionDuration` (default 30 days) — how long commit files are kept
- `delta.deletedFileRetentionDuration` (default 7 days) — how long removed data files are kept

OPTIMIZE

Compacts small files into larger ones (bin-packing). Doesn't change data, and is idempotent.

```
OPTIMIZE cat.schema.table;
OPTIMIZE cat.schema.table WHERE event_date >= '2026-08-01';
```

Z-Order (legacy) vs Liquid Clustering (current)

Z-Order co-locates related data using a space-filling curve so file-skipping statistics become effective on those columns.

```
OPTIMIZE t ZORDER BY (country, product_id);
```

Limitations: works alongside partitioning rather than replacing it, rewrites data every run, effective for only 1–4 columns, and **the columns cannot be changed** without rebuilding the table.

Liquid Clustering replaces both partitioning and Z-Order.

```
CREATE TABLE t (...) CLUSTER BY (country, event_date);
ALTER TABLE t CLUSTER BY (region, event_date); -- keys CAN change later
CREATE TABLE t (...) CLUSTER BY AUTO;          -- Databricks picks from query history
```

Advantages: clustering keys are changeable, clustering is incremental (OPTIMIZE only processes new data), it handles skewed key distributions, and it avoids the over-partitioning small-file trap. Mutually exclusive with partitioning and Z-Order.

VACUUM

Physically deletes data files that are no longer referenced by the current version **and** are older than the retention threshold.

```
VACUUM t RETAIN 168 HOURS; -- the unit is HOURS
VACUUM t DRY RUN;         -- list what would be deleted
```

The critical distinction: VACUUM controls file history and time-travel depth. It is *not* business data retention. Keeping five years of rows is a scheduled **DELETE** on the date column. **VACUUM ... RETAIN 43800 HOURS** (5 years) simply means you never reclaim storage.

The 7-day default exists because a long-running reader may still be reading logically removed files; deleting them mid-read causes **FileNotFoundException**.

spark.databricks.delta.retentionDurationCheck.enabled=false bypasses the guard — only for supervised one-off cleanups.

MERGE

Upsert in one statement. Internally: a join to find matches, then a rewrite of every file containing a matched row.

```
MERGE INTO target t
USING source s
  ON t.event_date = s.event_date -- partition/cluster column enables pruning
 AND t.id = s.id
WHEN MATCHED AND s.updated_at > t.updated_at THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
WHEN NOT MATCHED BY SOURCE AND t.event_date >= '2026-08-01' THEN DELETE;
```

Performance depends almost entirely on pruning. Put the partitioning or clustering column in the **ON** clause, pre-filter the target, and shrink and deduplicate the source. MERGE fails outright if one target row matches multiple source rows.

Deletion vectors

Instead of rewriting an entire Parquet file to remove some rows, Delta writes a bitmap marking deleted row positions ("merge-on-read"). DELETE, UPDATE, and MERGE become far faster. Files are physically cleaned up on the next OPTIMIZE.

```
ALTER TABLE t SET TBLPROPERTIES (delta.enableDeletionVectors = true);
```

Change Data Feed (CDF)

Records row-level changes as a queryable feed.

```
ALTER TABLE t SET TBLPROPERTIES (delta.enableChangeDataFeed = true);  
SELECT * FROM table_changes('cat.schema.t', 120, 135);
```

Adds `_change_type` (`insert`, `update_preimage`, `update_postimage`, `delete`), `_commit_version`, `_commit_timestamp`. This is the correct way to propagate updates and deletes downstream. Only captures changes from the point it is enabled.

Schema enforcement and evolution

Enforcement is default — a mismatched write fails. Evolution is opt-in:

- `mergeSchema` — add new columns on append
- `overwriteSchema` — replace the schema on overwrite
- `spark.databricks.delta.schema.autoMerge.enabled` — allow MERGE to add columns
- **Column mapping** (`delta.columnMapping.mode = 'name'`) — decouples logical names from physical Parquet names; **required to rename or drop a column**

Enabling column mapping raises the protocol version, so older clients can no longer read the table.

Other features

Feature	What it does
Generated columns	Computed on write; enables partition pruning from a filter on the source column
Identity columns	<code>GENERATED ALWAYS AS IDENTITY</code> — unique increasing integers, not gap-free
Constraints	<code>NOT NULL</code> and <code>CHECK</code> — violating writes fail the transaction
Shallow clone	Metadata-only copy pointing at the same files. Instant, near-free. Breaks if source is VACUUMed
Deep clone	Full copy, independent, incremental on re-run. Use for backups
UniForm	Writes Iceberg/Hudi metadata alongside so non-Delta engines read the same files
REPLACE WHERE	Atomically replaces only rows matching a predicate

Managed vs external tables

	Managed	External
Storage location	UC controls it	You specify <code>LOCATION</code>
<code>DROP TABLE</code>	Deletes the data (7-day UNDROP under UC)	Removes metadata only
Optimisation	Predictive optimisation, automatic maintenance	Manual

Prefer managed under Unity Catalog. Use external when another system also writes those files, or during a migration.

4. Medallion Architecture

What it is

A layering convention for a lakehouse — Bronze, Silver, Gold — where each layer has a defined purpose and data flows one direction.

Bronze — raw

Append-only, source fidelity preserved. Original columns plus ingest metadata: `_ingest_ts`, `_source_file`, `_batch_id`, `_rescued_data`.

Do not clean or deduplicate here. Bronze is the replay source of truth. If you clean it, you can never correctly reprocess.

Silver — conformed

Where the engineering happens: parsing, type casting, deduplication, conformed business keys, joins to reference data, quality rules, SCD handling. This is the reusable layer other teams build on.

Gold — business-facing

Aggregated and denormalised. Star schemas, KPI tables, feature tables. Optimised for read — clustered on whatever the dashboards filter by.

Why the separation earns its keep

Because Bronze is immutable and the transformations are deterministic, any window can be rebuilt with `replaceWhere` and a rerun. That's the recovery mechanism when a bad load propagates downstream.

Idempotency

A rerun must produce the same result. Achieved by:

- MERGE on a business key rather than blind append
- `replaceWhere` for a bounded window rather than full overwrite
- Deterministic surrogate keys (hashes, not sequences or random values)
- No `current_timestamp()` in business logic — pass the logical run date as a parameter
- `txnAppId` / `txnVersion` for streaming appends

5. Structured Streaming

What it is

Stream processing expressed with the same DataFrame API as batch. Conceptually, the stream is an unbounded input table; each trigger appends new rows; your query is logically re-run over the whole table, and the engine computes only the incremental change.

Micro-batch vs continuous

Micro-batch processes data in small batches with latency from ~100 ms to seconds, and gives full fault tolerance and exactly-once semantics. Continuous processing offers ~1 ms latency with at-least-once and very limited operator support — experimental, rarely used in production.

Triggers

Trigger	Behaviour	Use for
Default (unspecified)	Next batch as soon as the previous finishes	Continuous low latency
<code>processingTime="5 minutes"</code>	Fixed interval	Predictable near-real-time
<code>availableNow=True</code>	Process all available data in multiple batches, then stop	Batch-style incremental on a schedule
<code>continuous="1 second"</code>	Long-running tasks, at-least-once	Experimental

`availableNow` is the modern way to run scheduled incremental processing. You keep checkpoint-based tracking but pay only for the time the job runs, rather than an always-on cluster.

Output modes

Mode	Emits	Notes
Append	Only new rows, never updated	Default. Aggregations require a watermark
Update	Only rows changed this trigger	Works without a watermark, but state grows
Complete	The entire result table every trigger	Unbounded state — rarely appropriate

Checkpoints

The mechanism that makes streaming fault-tolerant and exactly-once.

```
<checkpointLocation>/
├─ offsets/      ← write-ahead log: what each batch WILL process
├─ commits/      ← what each batch DID complete
├─ sources/      ← source metadata (Auto Loader's seen-file RocksDB, etc.)
├─ state/        ← state store for stateful operations
└─ metadata/     ← query ID
```

The ordering is the mechanism. Offsets are written *before* processing; the commit is written *after*.

On restart, any batch with an offset entry but no commit entry is replayed with identical offsets. Deterministic replay plus an idempotent sink equals exactly-once.

One checkpoint per query. Never share.

Deleting the checkpoint loses all memory of processed data and all state. Depending on `startingOffsets`, a restart either skips everything in the gap or reprocesses from the beginning.

Checkpoint compatibility with code changes

Safe: adding filters or projections, adding output columns, changing trigger interval, changing cluster size, changing `maxFilesPerTrigger`.

Unsafe: changing the source, changing aggregation keys, adding or removing a stateful operator, changing output mode, changing the watermark column, changing join type.

The state store is keyed by operator ID and layout, so stateful shape changes either throw on restart or produce wrong results. Rule of thumb: stateless changes are fine, stateful shape changes are not.

Watermarks

A watermark declares how late you are willing to accept data:

```
df.withWatermark("event_ts", "10 minutes")
```

This lets the engine finalise and evict old state. Without it, aggregations and deduplication retain every key forever — unbounded growth ending in OOM.

Records arriving later than the watermark are **silently dropped**. Monitor `numRowsDroppedByWatermark`; if it climbs, the watermark is too tight and real data is being lost.

In stream-stream joins, both sides need watermarks plus a time-range condition so the engine can bound buffering. For outer joins, the engine cannot emit a null-padded row until the watermark passes — so outer results are delayed by the watermark interval by design.

State store

The default HDFS-backed store keeps state on the JVM heap, so large state causes GC thrash and OOM. **RocksDB** stores it off-heap on local disk with an in-memory cache.

```
spark.conf.set("spark.sql.streaming.stateStore.providerClass",
  "org.apache.spark.sql.execution.streaming.state.RocksDBStateStoreProvider")
spark.conf.set(
  "spark.sql.streaming.stateStore.rocksdb.changelogCheckpointing.enabled", "true")
```

Changelog checkpointing uploads deltas rather than full snapshots, substantially shortening batch times for large state.

Deduplication

`dropDuplicates(["id"])` in a stream retains every key ever seen.

`dropDuplicatesWithinWatermark(["id"])` with a watermark bounds state and evicts old keys. Always use the latter in streaming.

Arbitrary stateful processing

`flatMapGroupsWithState` and the newer `transformWithState` give you all events for a key, a mutable state object, and timers — you decide what to emit. Classic use: sessionization (close a session after 30 minutes of inactivity), fraud pattern detection.

foreachBatch

The streaming writer supports only append, complete, and update — it cannot MERGE. `foreachBatch` hands you the micro-batch as an ordinary DataFrame plus a batch ID, so you can do anything, including a Delta MERGE.

```
def upsert(batch_df, batch_id):
    batch_df = dedupe(batch_df)          # MERGE requires unique source keys
    (DeltaTable.forName(spark, "cat.silver.t").alias("t")
     .merge(batch_df.alias("s"), "t.id = s.id AND t.event_date = s.event_date")
     .whenMatchedUpdateAll(condition="s.updated_at > t.updated_at")
     .whenNotMatchedInsertAll().execute())
```

For plain appends inside `foreachBatch`, use `txnAppId` and `txnVersion` with the batch ID so Delta skips an already-committed batch.

Streaming from a Delta table

Streaming Delta sources expect append-only data. A MERGE upstream rewrites files, which looks like non-append change and throws.

Option	Effect
<code>ignoreDeletes</code>	Skip commits that only delete whole partitions
<code>ignoreChanges</code>	Re-emit rewritten files — produces duplicates downstream
<code>skipChangeCommits</code>	Skip update/delete commits entirely (cleaner, newer)
<code>startingVersion</code> / <code>startingTimestamp</code>	Where to begin
<code>maxFilesPerTrigger</code> / <code>maxBytesPerTrigger</code>	Throughput control

For propagating updates and deletes properly, use **Change Data Feed** rather than `ignoreChanges`.

Monitoring

`StreamingQueryProgress` exposes `inputRowsPerSecond`, `processedRowsPerSecond`, `batchDuration`, `numInputRows`, `stateOperators[].numRowsTotal` (state size), and offset lag. When `processedRowsPerSecond` is below `inputRowsPerSecond`, you are falling behind permanently.

6. Auto Loader

What it is

An incremental file-ingestion source (`cloudFiles`) that processes only newly arrived files, without re-

listing the entire directory each trigger.

Why it exists

Plain file streaming lists the whole directory every trigger, so cost grows with the total number of files ever written rather than the new ones. At high file counts this becomes the dominant cost.

How it tracks files

A RocksDB store inside the checkpoint (`<checkpoint>/sources/`), keyed by file path. This is why deleting the checkpoint means reprocessing everything.

Discovery modes

Directory listing (default) — lists the path each trigger. Simple, no extra cloud permissions. Fine up to tens of thousands of files.

File notification — Auto Loader provisions cloud resources (Azure: Event Grid subscription + Storage Queue; AWS: SNS topic + SQS queue) and consumes events. Cost and latency stay roughly constant regardless of directory size. Requires permission to create those resources, or pre-created ones passed by name.

File events — the newer UC-managed approach, configured on the external location rather than per stream.

Always set `cloudFiles.backfillInterval` (e.g. `"1 day"`) so a periodic full listing catches files whose notifications were dropped.

Schema handling

`cloudFiles.schemaLocation` stores the inferred schema. By default JSON and CSV columns are inferred as strings to avoid type drift; `inferColumnTypes=true` gives real types. **Schema hints** override specific columns without abandoning inference.

Evolution modes

Mode	Behaviour
<code>addNewColumns</code> (default)	Stream fails on a new column, records the new schema, retry picks it up
<code>rescue</code>	Schema never changes; unexpected fields go to <code>_rescued_data</code> . Stream never stops
<code>failOnNewColumns</code>	Fails and does not update the schema
<code>none</code>	Ignores new columns silently

The rescued data column

A JSON string capturing every field that didn't match the schema, plus the source file path. Nothing is silently lost. For unpredictable sources this is the safety net — query it later to discover new fields and promote them deliberately.

7. Kafka and Event Streaming

The model

Kafka is a distributed append-only log. Producers write to **topics**, which are split into **partitions**. Ordering is guaranteed **within a partition only** — global ordering is not achievable at scale.

How Spark reads it

Spark does **not** use Kafka consumer groups. It manages offsets itself in the checkpoint using assign semantics, and never commits to Kafka's `__consumer_offsets`.

The consequence: Kafka-side consumer lag monitoring will show nothing for your stream. Monitor via `StreamingQueryProgress` instead.

Key options

Option	Purpose
<code>subscribe</code> / <code>subscribePattern</code> / <code>assign</code>	Which topics or partitions
<code>startingOffsets</code>	<code>earliest</code> , <code>latest</code> , or explicit JSON offsets
<code>maxOffsetsPerTrigger</code>	Backpressure
<code>minPartitions</code>	Split a Kafka partition's offset range across multiple tasks
<code>failOnDataLoss</code>	If false, silently skips missing offsets when retention expired

Parallelism defaults to one Spark task per Kafka partition, so adding executors past that count does nothing.

Payload parsing

Key and value arrive as binary. Always define an explicit `StructType` and use `from_json` , or use Avro with a schema registry via `from_avro` . Never infer schema on a streaming source — inference is non-deterministic across batches and will change your output schema mid-stream.

Azure Event Hubs

Connect via the Kafka-compatible endpoint (`<namespace>.servicebus.windows.net:9093` , SASL_SSL with PLAIN, connection string from a secret scope). Differences from Kafka: throughput is capped by Throughput/Processing Units, retention is shorter by default, and there's no Kafka Connect ecosystem. Event Hubs Capture can auto-land raw events to storage, which pairs well with Auto Loader.

8. Databricks Platform

Compute types

Type	Lifecycle	Cost	Use for
All-purpose	Manual start, auto-terminate on idle	Highest DBU rate	Development, ad-hoc notebooks
Job cluster	Created at job start, destroyed at end	~half all-purpose	All scheduled production work
SQL Warehouse	Auto start/stop on query	SQL DBU rate	BI tools, dashboards
Serverless SQL	Starts in seconds	Highest per-second, zero idle	Bursty BI, short queries
Serverless jobs	No cluster to manage	Per-second	Short jobs
Instance pool	Pre-warmed idle VMs	Cloud VM cost while idle, no DBU	Cutting 5-minute startup to ~30s

Using an all-purpose cluster for a scheduled job is the most common cost mistake — roughly double the rate, plus idle time between runs.

Access modes

Mode	Users	Unity Catalog	Notes
Dedicated	One user or group	Full	Required for Scala and some ML/streaming libraries
Standard	Multiple concurrent	Full	Process isolation; restricts <code>sparkContext</code> , RDD APIs, mounts
No-isolation shared	Multiple	None	Legacy. Avoid

Standard-mode restrictions are the usual blocker when migrating older code — RDD-based jobs must be rewritten against the DataFrame API.

Databricks Runtime (DBR)

Versioned major.minor (14.3, 15.4, 16.x). **LTS** releases get roughly three years of support — pin production to LTS and keep all environments on the same version.

Flavours: standard, **ML** (adds PyTorch, TensorFlow, MLflow, scikit, plus GPU variants), and **Photon-enabled**.

Each DBR bundles specific Spark, Delta, Python, and library versions, so upgrading can silently change a dependency. Regression-test before bumping.

Photon

A vectorized C++ query engine replacing parts of Spark's JVM execution. Accelerates SQL and DataFrame operations, joins, aggregations, and Delta writes. **Falls back to Spark** for Python and Scala UDFs, RDD operations, and some complex types.

Costs roughly 2× DBUs per hour, so evaluate on total job cost (rate × runtime), not hourly rate. Wins

on SQL and Delta-heavy work; loses on UDF-heavy Python.

Instance families

Family	Azure	AWS	vCPU:RAM	Use when
General purpose	D-series	m5d , m6gd	1:4	Default ETL
Memory optimized	E-series	r5d , r6gd	1:8	Wide aggregations, big joins, spill
Compute optimized	F-series	c5d	1:2	CPU-bound parsing, Photon SQL
Storage optimized	L-series	i3 , i4i	1:8 + NVMe	Shuffle-heavy, disk caching
GPU	NC/ND	p3 , g5	—	Deep learning only

Always choose a type with **local SSD/NVMe** (the **d** in **D8ds_v5** , **m5d**) — shuffle spill and the Delta disk cache write to local disk.

Autoscaling

Adds workers when tasks are pending, removes them after idleness. Scale-up is aggressive; scale-down is conservative because it won't drop an executor holding needed shuffle data.

Poor fit for streaming — continuous micro-batches keep executors busy so it rarely scales down, and scale-down forces state redistribution. Use a fixed size and control load with backpressure instead.

Poor fit for short jobs — VM acquisition takes minutes, so the job often finishes before help arrives.

Spot instances

Up to 70–90% cheaper but reclaimable at any time. Safe for batch with retries and dev clusters; risky for long-running stateful streaming and tight SLAs. Best practice: on-demand driver plus spot workers, with a **first_on_demand** count.

Cluster policies

JSON rule sets constraining what users can create — allowed node types, max workers, max DBU/hour, mandatory auto-termination, mandatory tags, fixed DBR version, forced access mode. Rule types include **fixed** , **allowlist** , **blocklist** , **range** , **regex** .

Jobs and Workflows

Multi-task DAGs with dependencies, conditional (**if/else**) tasks, **for each** loops, retries, timeouts, and notifications. Task types include notebook, Python wheel, JAR, SQL, dbt, DLT pipeline, and Run Job. Values pass between tasks via **dbutils.jobs.taskValues** . Triggers can be scheduled, continuous, or **file arrival** .

Databricks Asset Bundles (DABs)

Job definitions, cluster specs, and code declared in YAML alongside the code, deployed per environment with **databricks bundle deploy -t prod** . Environment differences (catalog name, service principal) are variables, not hardcoded strings.

Cost model

Total cost = DBU rate × runtime + underlying cloud VM cost. The second half is frequently forgotten. DBU rate varies by workload type, pricing tier, and Photon.

Attribution requires **tags**, enforced through cluster policies, then queried from `system.billing.usage`.

9. Unity Catalog

What it is

Databricks' governance layer for data and AI assets — a central catalog that sits above workspaces and provides identity, access control, lineage, audit, and sharing.

Why it exists

The Hive metastore was per-workspace, two-level (`schema.table`), and had no real access control. Data was reached through **mounts**, which stored credentials at the workspace level — so anyone with cluster access inherited them. That completely bypasses per-user authorization.

The object hierarchy

```
Account
├── Metastore (one per region)
│   ├── Catalog
│   │   ├── Schema (database)
│   │   │   ├── Table (managed | external)
│   │   │   ├── View / Materialized view
│   │   │   ├── Volume (managed | external) ← non-tabular files
│   │   │   ├── Function
│   │   └── Model (MLflow registered model)
└── Account-level objects:
    ├── Storage credentials · External locations · Connections (federation)
    └── Shares / Recipients (Delta Sharing)
```

The three-level namespace (`catalog.schema.table`) is what allows clean environment or domain separation — `dev_sales` and `prod_sales` as genuinely separate catalogs rather than naming conventions.

One metastore per region. A multi-region client needs one per region; cross-region access happens via Delta Sharing or replication. This also matters for data residency and GDPR.

Storage credentials and external locations

A deliberate separation of concerns:

- **Storage credential** = the *identity*. Azure: managed identity via an Access Connector. AWS: an IAM role with a trust policy for the Databricks account.

- **External location** = a specific cloud path **plus** the credential to use for it.

Users are granted `READ FILES` / `WRITE FILES` on the external location, or work through tables and volumes built on top. Admins manage the credential once; users never see it.

Volumes

UC-governed storage for non-tabular files — JSON landing zones, images, model artifacts, libraries, init scripts. Accessed as `/Volumes/catalog/schema/volume/path`. Managed volumes are UC-controlled; external volumes point at an existing cloud path.

Volumes are the replacement for DBFS mounts.

Privileges

`USE CATALOG`, `USE SCHEMA` (traversal — required to reach anything below), `SELECT`, `MODIFY`, `CREATE TABLE`, `CREATE SCHEMA`, `READ VOLUME`, `WRITE VOLUME`, `EXECUTE`, `APPLY TAG`, `BROWSE`, `ALL PRIVILEGES`.

Privileges **inherit downward** — a grant on a catalog covers every table within it.

Owner ≠ grantee. Only the owner or a metastore admin can DROP, ALTER, or manage grants. Best practice: own objects with a **group or service principal**, never an individual, so nothing breaks when someone leaves.

Row filters and column masks

Fine-grained access control attached to the table itself, so consumers query the real table name.

```
CREATE FUNCTION sec.mask_pii(v STRING)
RETURN CASE WHEN is_account_group_member('pii_readers') THEN v ELSE '***' END;

ALTER TABLE cat.silver.customers ALTER COLUMN phone SET MASK sec.mask_pii;

CREATE FUNCTION sec.region_filter(region STRING)
RETURN is_account_group_member('global_admins') OR region = current_user_region();

ALTER TABLE cat.silver.customers SET ROW FILTER sec.region_filter ON (region);
```

This replaces the older pattern of maintaining separate dynamic views per audience — copies drift, and views multiply until nobody knows which is authoritative.

Lineage

Table- and column-level lineage captured automatically from queries run on UC-enabled compute, retained about a year, visible in Catalog Explorer and queryable via `system.access.table_lineage` and `column_lineage`. Not captured for operations that bypass UC.

Delta Sharing

An open protocol for sharing live data without copying.

- **Databricks-to-Databricks** — the recipient sees a shared catalog; no credential exchange.
- **Open sharing** — the recipient gets a credential file and uses pandas, Spark, Power BI, or Snowflake connectors. Works for non-Databricks consumers.

The recipient always reads current data, and access can be revoked instantly.

Lakehouse Federation

A **CONNECTION** to an external system (SQL Server, Postgres, Snowflake, Redshift, MySQL, BigQuery) surfaced as a **foreign catalog** inside UC. Queried with the same three-level namespace and governed by UC, without ingesting. Pushdown is limited, so it suits exploration and small dimension joins rather than heavy production ETL.

System tables

Table	Use
<code>system.access.audit</code>	Every action, who and when — security and compliance
<code>system.access.table_lineage</code> / <code>column_lineage</code>	Dependency tracing, impact analysis
<code>system.billing.usage</code> + <code>list_prices</code>	Cost attribution by tag, cluster, SKU
<code>system.compute.clusters</code>	Cluster configuration history
<code>system.query.history</code>	Slow queries on SQL warehouses
<code>system.information_schema.*</code>	Table and column metadata, grants

Admin roles

Account admin manages the account, metastores, and identities. **Metastore admin** owns the metastore and can manage all objects in it. **Workspace admin** manages a workspace's clusters and jobs but is *not* automatically granted data access — that separation of duty is intentional.

Migrating from Hive metastore and mounts

1. Inventory mounts and the tables using them
2. Create a storage credential, then an external location on the same underlying path
3. Files → external volume; change code from `/mnt/...` to `/Volumes/...`
4. Tables → **SYNC TABLE** (creates a UC external table over the same files, so both names work simultaneously — this is what gives zero downtime), or CTAS into managed tables
5. Grant privileges to groups
6. Verify, then remove the mount

UCX is Databricks' open-source toolkit for automating assessment and migration at scale.

10. Delta Live Tables / Lakeflow Declarative Pipelines

What it is

A declarative framework: you define tables and their dependencies, and the framework infers the DAG, provisions compute, manages checkpoints and retries, applies quality rules, tracks lineage, and

maintains an event log.

Dataset types

Type	Semantics	Use for
Streaming table	Incremental, append-oriented; each input row processed once	Ingestion (Bronze), append-only Silver
Materialized view	Query result, recomputed (incrementally where possible) on refresh	Aggregations, Gold
View	Not materialized, computed at query time	Intermediate logic

Expectations

Declarative data quality rules with per-rule severity:

```
@dlt.expect("valid_ts", "event_ts IS NOT NULL")           # log only
@dlt.expect_or_drop("valid_id", "id IS NOT NULL")          # drop the row
@dlt.expect_or_fail("no_negative", "amount >= 0")         # fail the pipeline
```

Violation counts land in the event log, so quality dashboards come from the framework rather than bolt-on monitoring.

APPLY CHANGES INTO

CDC and SCD handling without hand-written MERGE:

```
APPLY CHANGES INTO live.dim_customer
FROM STREAM(live.bronze_customer)
KEYS (cust_id)
APPLY AS DELETE WHEN op = 'D'
SEQUENCE BY seq_num
STORED AS SCD TYPE 2;
```

SEQUENCE BY handles out-of-order arrival correctly.

Modes

Development reuses the cluster and skips retries so errors surface immediately. **Production** uses a fresh cluster per run with full retry semantics.

Triggered runs, processes available data, and stops. **Continuous** stays on for low latency at always-on cost.

Full refresh

Reprocesses everything and clears state. Dangerous when the source has retention limits — a Kafka source with 7-day retention cannot supply months of history. Protect critical tables with `pipelines.reset.allowed = false`.

When not to use DLT

When you need fine-grained cluster control, complex non-declarative orchestration (external API calls, conditional cross-system branching), or when the team already has mature Workflows plus Asset Bundles tooling. DLT trades control for velocity.

11. Data Modeling

Star vs snowflake

Star — one fact table joined directly to denormalised dimensions. Fewer joins, faster, and the default for a lakehouse. **Snowflake** — dimensions normalised into sub-dimensions; saves storage, adds joins. Since columnar storage is cheap and joins cost shuffle, prefer star.

Fact table grain

The grain is what one row means ("one line item on one invoice"). Define it before anything else — every dimension and measure must be consistent with it. Mixing grains is the root cause of most double-counting bugs.

Measure types

Additive — summable across all dimensions (revenue, quantity). **Semi-additive** — summable across some but not time (account balance, inventory); average or take the last value over time. **Non-additive** — never summable (ratios, percentages); store numerator and denominator instead.

Slowly Changing Dimensions

Type	Behaviour
0	Never changes (date of birth)
1	Overwrite, no history
2	New row per change with <code>effective_from</code> , <code>effective_to</code> , <code>is_current</code>
3	Add a <code>previous_value</code> column — one level of history
4	Current table plus separate history table
6	Hybrid 1+2+3

Type 2 in Delta requires two-sided staging, because one MERGE cannot both close the old row and insert a new one for the same key. Stage each changed record twice — once with the real key (matches, closes the old row) and once with a null merge key (never matches, falls through to insert).

Surrogate keys

`monotonically_increasing_id()` is unique but **not sequential or stable** — it encodes the partition ID, so values change if partitioning changes. Never persist it.

Hash keys (`sha2(concat_ws('||', natural_keys), 256)`) are deterministic and reproducible across reruns, and they solve late-arriving dimensions — a fact can compute the dimension's key itself and load immediately.

Identity columns give real integers but serialise writes somewhat and are not gap-free.

Late-arriving dimensions

A fact references a dimension row that doesn't exist yet. Options: insert an **inferred member** placeholder and update it later; quarantine the fact for retry; or use deterministic hash keys so the fact loads immediately and the dimension backfills. The third is cleanest in a lakehouse.

Load patterns

Pattern	When	Limitation
Full	Small reference data, or no reliable change marker	Doesn't scale
Incremental (watermark)	Reliable <code>modified_ts</code> , soft deletes only	Misses hard deletes
CDC	Hard deletes matter, ordering matters	Most complex

The classic failure is incremental loading against a source that hard-deletes — deleted rows quietly persist in the warehouse forever.

Partitioning strategy

A good partition column is low-to-medium cardinality, frequently filtered, and evenly distributed — usually a date. High cardinality creates millions of directories with tiny files.

Rule of thumb: each partition should hold at least ~1 GB. Under about 1 TB total, consider not partitioning at all and using **Liquid Clustering** instead.

12. File Formats

Format	Type	Pushdown	Use for
Parquet	Columnar	Yes	Analytics default
ORC	Columnar	Yes	Hive ecosystem
Avro	Row-based	No	Streaming payloads, strong schema evolution
JSON	Row-based text	No	Landing zone only
CSV	Row-based text	No	Interchange only
Delta	Parquet + log	Yes	Tables — ACID, time travel, MERGE

Predicate pushdown evaluates filters at the file-format level so fewer rows are read. **Column pruning** reads only requested columns. Both work with Parquet, ORC, and Delta; neither works with CSV or JSON, which must be parsed in full.

13. Data Quality and Governance

Checks by layer

Bronze asks *did we receive it* — file arrived, row count above zero, ingest metadata populated, rescued data not filling unexpectedly.

Silver asks *is it correct* — not-null keys, uniqueness on the business key, referential integrity, valid domains, sensible ranges.

Gold asks *does it reconcile* — aggregates tie back to Silver, no nulls in KPI columns, period-over-period variance within threshold.

Quarantine over failure

Failing a whole job because a handful of rows are bad means good data doesn't reach the business. Route invalid rows to a quarantine table with a reason and timestamp, keep the pipeline moving, and make the bad data visible and reprocessable. Fail hard only when the data is fundamentally untrustworthy.

Reconciliation

Write per-batch metrics to a control table: source rows, Bronze rows, Silver rows after dedup, rows quarantined. The invariant `bronze = silver + duplicates_removed + quarantined` should hold; alert when it doesn't.

Right to be forgotten

More than a DELETE. Delete from every table holding the subject (requires a registry of where PII lives), then VACUUM — until VACUUM runs past retention, the rows are still physically present and recoverable by time travel. Also consider Change Data Feed, old checkpoints, and clones. **Crypto-shredding** (per-subject encryption keys, destroy the key) is the alternative when physical deletion across all copies is impractical, but it must be designed upfront.

Disaster recovery

RPO and RTO drive the design. `DEEP CLONE` to a secondary region on a schedule gives a consistent point-in-time copy, incremental after the first run. Code lives in Git; job definitions in Asset Bundles; UC metadata needs a documented recreation plan since the metastore is regional. An untested restore is not a plan.

14. Cloud Foundations

Azure

ADLS Gen2 vs Blob — Gen2 adds a hierarchical namespace: real directories, atomic renames, POSIX-like ACLs. Without it, renaming a directory copies every object, which makes Spark commits slow and non-atomic.

Access from Databricks — create an Access Connector (managed identity), grant it Storage Blob Data Contributor, register it as a UC storage credential, create an external location. No keys, no mounts, no secrets in code.

Secret scopes — Databricks-backed or Key Vault-backed. Read with `dbutils.secrets.get(scope, key)`; values are redacted in output.

ADF — linked service (connection), dataset (shape), activity (unit of work), pipeline (DAG), integration runtime (Azure IR for cloud, **self-hosted IR** for on-prem/VNet sources). Triggers: schedule, **tumbling window** (fixed windows with backfill and dependency chaining — the right choice for date-partitioned incremental loads), and event-based.

Networking — VNet injection, Private Endpoints to keep traffic off the public internet, NSGs and UDRs for egress control, NCC for stable serverless outbound identities.

AWS

S3 + IAM — UC uses an IAM role with a trust policy for the Databricks account, registered as a storage credential. Instance profiles are the legacy per-cluster equivalent and don't support per-user authorization. S3 has been strongly consistent since 2020.

Glue vs Unity Catalog — Glue is Hive-compatible metadata with coarse IAM-level control and no lineage. UC adds three-level namespacing, fine-grained grants, lineage, audit, volumes, and sharing.

Storage classes — do not lifecycle-tier Delta data files to Glacier while still referenced by the log; reads will fail. Use **DEEP CLONE** to move a snapshot instead.

15. Glossary

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability — transaction guarantees
AQE	Adaptive Query Execution — runtime re-optimisation using real statistics
Auto Loader	Incremental file ingestion source (cloudFiles)
Bronze/Silver/Gold	Medallion layers: raw / conformed / business-facing
Catalyst	Spark's query optimiser
CDC	Change Data Capture — reading a source database's change log
CDF	Change Data Feed — Delta's own row-level change record
Checkpoint (streaming)	Offsets, commits, and state enabling fault-tolerant restart
DBR	Databricks Runtime
DBU	Databricks Unit — the billing unit for compute
Deletion vector	Bitmap marking deleted rows instead of rewriting files
DLT	Delta Live Tables / Lakeflow Declarative Pipelines
Executor	JVM process on a worker node running tasks
External location	A cloud path plus the storage credential used to access it
Grain	What one row of a fact table represents
Idempotency	A rerun produces the same result
Liquid Clustering	Changeable, incremental data layout replacing partitioning and Z-order
Medallion	The Bronze/Silver/Gold layering convention
MERGE	Upsert statement combining insert, update, and delete

Metastore	The top-level UC container, one per region
OPTIMIZE	Compacts small Delta files
Partition	A chunk of data; the unit of parallelism
Photon	Vectorized C++ execution engine
Predicate pushdown	Evaluating filters at the file-format level
Rescued data	Auto Loader column capturing fields that didn't match the schema
SCD	Slowly Changing Dimension
Shuffle	Redistributing data across partitions by key
Skew	Uneven key distribution causing one task to dominate runtime
Spill	Writing intermediate data to disk when memory is exhausted
Storage credential	The identity UC uses to authenticate to cloud storage
Time travel	Querying a previous version of a Delta table
Tungsten	Spark's memory and CPU efficiency layer
Unity Catalog	Databricks' governance layer for data and AI assets
VACUUM	Physically deletes unreferenced Delta files past a retention threshold
Volume	UC-governed storage for non-tabular files
Watermark	Declared tolerance for late-arriving event-time data
Z-Order	Legacy multi-dimensional clustering via space-filling curve